DESENVOLVIMENTO WEB COM .NET

**ANÁLISE E DESENVOLVIMENTO DE SISTEMAS**

# RELATÓRIO DE AULAS PRÁTICAS

Relatório 4 – Recursos REST, paginação, filtros, Problem Details, OpenAPI e cliente TypeScript

Nome: _____

RA: _____

Polo de matrícula: _____

Local da realização da Aula Prática: _____

Ano da postagem: 2026

| | | |
|---|---|---|
| **Instituto de Ciências Exatas e Tecnologia** | **Disciplina:** Desenvolvimento Web com .NET<br>**Título da Aula:** Recursos REST, paginação, filtros, Problem Details, OpenAPI e cliente TypeScript | **Relatório 4** |

**Unidade 4 – APIs REST e OpenAPI**

**Objetivo:** Expor o domínio de tarefas como API HTTP previsível, documentar o contrato e gerar um cliente TypeScript reproduzível.

## 1. Identificação e ambiente

Repositório: https://github.com/maiaralopes-dev/projetos (branch `ccr-836bb896-v5ejij`, pasta `PortalTarefas/`); commit `65c1594`. Pacotes: `Microsoft.AspNetCore.OpenApi` 10.0.14 (`AddOpenApi`/`MapOpenApi`, OpenAPI 3.1), `Microsoft.Extensions.ApiDescription.Server` 10.0.14 (gera o documento no build), `Scalar.AspNetCore` 2.19.0 (UI interativa só em Development), **NSwag 14.7.1** (npm, runtime Net100), Node 22.23.4 e TypeScript 7.0.

## 2. Resumo teórico aplicado ao projeto

**Recursos, URIs e métodos.** O recurso é a coleção `/api/v2/tarefas` e cada item `/api/v2/tarefas/{id}`. GET lê (seguro e idempotente), POST cria (não idempotente; responde **201** com `Location`), PUT substitui o item (idempotente; 200), DELETE remove (idempotente; 204 e depois 404). Erros: 400 (entrada inválida), 404 (inexistente), 409 (conflito de concorrência ou título duplicado).

**Separação de modelos.** A entidade `Tarefa` (com `Versao` binária, auditoria e navegações) nunca é serializada. A API usa `CriarTarefaDto` e `AtualizarTarefaDto` (entrada), `TarefaResumoDto` e `TarefaDetalhesDto` (saída) e `PaginaDto<T>` (representação paginada com `itens`, `pagina`, `tamanho`, `totalItens`, `totalPaginas`).

**Paginação, ordenação e filtros.** `pagina` ≥ 1 (padrão 1), `tamanho` de 1 a 50 (padrão 10); valores fora do limite são **rejeitados com 400** em vez de corrigidos silenciosamente. `ordenarPor` aceita só a **lista permitida** (`id`, `titulo`, `prazo`, `prioridade`, `situacao`, `criadaEm`) mapeada para expressões conhecidas — nunca para nomes de coluna vindos do usuário. Filtros: `situacao`, `prioridade`, `busca`, `prazoAte`. Tudo é aplicado no `IQueryable` antes do `ToListAsync`, ou seja, vira `WHERE/ORDER BY/OFFSET FETCH` no SQL.

**Problem Details.** Formato JSON padronizado (`type`, `title`, `status`, `detail`, `instance` + extensões) para que clientes tratem erros automaticamente. A **RFC 9457** substitui a RFC 7807 citada no plano. Extensões usadas: `errors` (validação), `correlationId`, `traceId` e `versaoAtual` (409 de concorrência) — nenhuma com dado sensível.

**OpenAPI × UI interativa.** O documento OpenAPI (`/openapi/v2.json`) é o **contrato** legível por máquina; Swagger UI, Scalar ou ReDoc são apenas visualizadores que leem esse documento. Por isso o documento é publicado em todos os ambientes, enquanto a UI Scalar só existe em Development.

**Cliente gerado e compatibilidade.** O cliente TypeScript é derivado do documento: se o contrato muda de forma incompatível (remover/renomear campo, mudar tipo, tornar

obrigatório), o código que usa o cliente deixa de compilar ou falha em execução; mudanças compatíveis (novo campo de saída, novo parâmetro opcional) só exigem regenerar.

## 3. Código-fonte e evidências

**Tabela 1** – Contrato definido antes da implementação

| Método e URI | Sucesso | Erros previstos |
|---|---|---|
| `GET /api/v2/tarefas?pagina&tamanho&ordenarPor&direcao&situacao&prioridade&busca&prazoAte` | 200 `PaginaTarefaResumo` + `X-Total-Count` + `Link` | 400 |
| `GET /api/v2/tarefas/{id}` | 200 `TarefaDetalhesDto` | 404 |
| `POST /api/v2/tarefas` | 201 + `Location` | 400, 409 (título duplicado) |
| `PUT /api/v2/tarefas/{id}` (com `versao`) | 200 `TarefaDetalhesDto` | 400, 404, 409 (`versaoAtual`) |
| `DELETE /api/v2/tarefas/{id}` | 204 | 404 |

**Revisão do documento OpenAPI** (etapa 3): o primeiro documento gerado tinha problemas corrigidos antes da entrega — (1) `pagina`/`tamanho` saíam como **obrigatórios** e em PascalCase → propriedades anuláveis com `[FromQuery(Name=...)]` e `[DefaultValue]`; (2) números como `type: [integer, string]` com *pattern* → `NumberHandling = Strict` no JSON; (3) nome genérico `PaginaDtoOfTarefaResumoDto` → `CreateSchemaReferenceId` gera `PaginaTarefaResumo`; (4) respostas 404/409 ausentes → `ProducesProblem(...)`; (5) `descricao` obrigatória no POST → parâmetro com valor padrão. Nulabilidade conferida: `descricao` e `prazo` como `["null","string"]`.

**Cliente TypeScript reproduzível:** o build do .NET grava `docs/openapi/portal-tarefas_v2.json` (artefato versionado); `npm run gerar-cliente` executa `nswag run nswag.json`, que gera `src/api/gerado/portal-tarefas-client.ts` (classe `PortalTarefasClient`, template Fetch, enums como *string literal*). Chamada real usando só o cliente gerado:

```
$ npm run chamada-real
Página 1/1 — 3 tarefas pendentes
  #6 Revisar SQL da migration (prazo 2026-11-08)
  #3 Escrever relatório 1 (prazo 2026-11-16)
  #9 Documentar API REST (prazo 2026-12-01)
Criada: 10 Criada pelo cliente TS 1794188188434 versão AAAAAAAAB9s=
Erro esperado (Problem Details): 404 Tarefa não encontrada.
```

Exemplos de resposta (saídas completas no Apêndice B): **sucesso** com cabeçalhos de paginação, **página vazia** (`pagina=99` → `itens: []`), **validação** (`tamanho=51` → 400 com `errors.Tamanho`) e **inexistente** (`/999` → 404).

## 4. Testes e validações obrigatórias

**Tabela 2** – Testes obrigatórios (14 testes de API e contrato, todos aprovados)

| Caso exigido | Resultado obtido | Evidência |
|---|---|---|
| Paginação com limites mínimo e máximo | `tamanho=0`, `tamanho=51`, `pagina=0` → 400; `tamanho=1` e `tamanho=50` → 200 | `01-requisicoes.txt`; 5 testes |
| Ordenação permitida e não permitida | `ordenarPor=prazo` ordena; `ordenarPor=versao` → 400 com a lista aceita | `Ordenacao_*` (2 testes) |
| Combinação de dois filtros | `situacao=Pendente&prioridade=Alta` e `situacao + prazoAte` retornam só itens que atendem a | `Combinacao_de_dois_filtros` |

| | | |
|---|---|---|
| | ambos | |
| Problem Details 400, 404 e 409 | 400 validação; 404 inexistente; 409 título duplicado e 409 versão antiga com `versaoAtual` | `Crud_completo_*`, `Post_invalido_*` |
| Regeneração após mudança compatível | Novo campo `vencida` → diff do OpenAPI e do cliente; `tsc` sem erros | `06-regeneracao-cliente.txt` |

## 5. Aprimoramentos

- **Teste de contrato (snapshot):** `Documento_openapi_corresponde_ao_snapshot_versionado` compara o documento servido com o artefato em `docs/openapi`; qualquer mudança não revisada no contrato quebra o teste.
- **Versionamento com mudança incompatível controlada:** a v1 (unidade 1) devolvia um *array*; a v2 devolve uma página. A v1 continua ativa, mas responde `Deprecation: @1798761600` (RFC 9745) e `Link: </api/v2/tarefas>; rel="successor-version"`.
- **Metadados de paginação** documentados: `X-Total-Count` e `Link` com `first`, `prev`, `next`, `last` (RFC 8288), preservando os demais filtros da URL.
- **Defeito real encontrado pelos testes:** um PUT repetido com a versão antiga e os mesmos valores retornava 200, pois o EF não emitia `UPDATE`; corrigido (ver Relatório 3).

## Referências

MICROSOFT. **Create responses in Minimal API applications**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/aspnet/core/fundamentals/minimal-apis/responses. Acesso em: 9 nov. 2026.

MICROSOFT. **Generate OpenAPI documents**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/aspnet/core/fundamentals/openapi/aspnetcore-openapi. Acesso em: 9 nov. 2026.

MICROSOFT. **Customize OpenAPI documents**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/aspnet/core/fundamentals/openapi/customize-openapi. Acesso em: 9 nov. 2026.

NOTTINGHAM, M.; WILDE, E.; DALAL, S. **RFC 9457: Problem Details for HTTP APIs**. IETF, 2023. Disponível em: https://www.rfc-editor.org/rfc/rfc9457. Acesso em: 9 nov. 2026.

NOTTINGHAM, M. **RFC 8288: Web Linking**. IETF, 2017. Disponível em: https://www.rfc-editor.org/rfc/rfc8288. Acesso em: 9 nov. 2026.

DALAL, S.; WILDE, E. **RFC 9745: The Deprecation HTTP Response Header Field**. IETF, 2025. Disponível em: https://www.rfc-editor.org/rfc/rfc9745. Acesso em: 9 nov. 2026.

OPENAPI INITIATIVE. **OpenAPI Specification v3.1.1**. 2024. Disponível em: https://spec.openapis.org/oas/v3.1.1. Acesso em: 9 nov. 2026.

SUTER, R. **NSwag**. GitHub, 2025. Disponível em: https://github.com/RicoSuter/NSwag. Acesso em: 9 nov. 2026.

## Apêndice A – Código-fonte integral

**Arquivo:** `src/PortalTarefas.Web/Endpoints/TarefasApiV2Endpoints.cs` (commit `65c1594`)

```
using Microsoft.AspNetCore.Http.HttpResults;
using Microsoft.AspNetCore.Mvc;
using PortalTarefas.Web.Contratos;
using PortalTarefas.Web.Modelos;
using PortalTarefas.Web.Servicos;

namespace PortalTarefas.Web.Endpoints;

/// <summary>
/// API REST v2 de tarefas. Contrato:
///   GET    /api/v2/tarefas          200 (página) | 400
///   GET    /api/v2/tarefas/{id}     200 | 404
///   POST   /api/v2/tarefas          201 + Location | 400 | 409 (título duplicado)
///   PUT    /api/v2/tarefas/{id}     200 | 400 | 404 | 409 (versão desatualizada ou título duplicado)
///   DELETE /api/v2/tarefas/{id}     204 | 404
/// </summary>
public static class TarefasApiV2Endpoints
{
    public const string TipoConflito = "https://portaltarefas.example/problemas/conflito";

    public static RouteGroupBuilder MapTarefasApiV2(this IEndpointRouteBuilder app)
    {
        var grupo = app.MapGroup("/api/v2/tarefas")
            .WithTags("Tarefas")
            .ProducesProblem(StatusCodes.Status500InternalServerError);

        grupo.MapGet("/", Listar).WithName("ListarTarefas")
            .WithSummary("Lista tarefas com filtros, ordenação e paginação.")
            .WithDescription("Ordenação aceita somente os campos da lista permitida. Os metadados de
paginação " +
                             "também são enviados nos cabeçalhos X-Total-Count e Link (RFC 8288).");
        grupo.MapGet("/{id:int}", Obter).WithName("ObterTarefa").WithSummary("Obtém os detalhes de uma
tarefa.")
            .ProducesProblem(StatusCodes.Status404NotFound);
        grupo.MapPost("/", Criar).WithName("CriarTarefa").WithSummary("Cadastra uma tarefa.")
            .ProducesProblem(StatusCodes.Status409Conflict);
        grupo.MapPut("/{id:int}", Atualizar).WithName("AtualizarTarefa")
            .WithSummary("Substitui os dados de uma tarefa.")
            .WithDescription("Envie em 'versao' o valor recebido no GET. Se outra alteração ocorreu
depois, a resposta é 409 " +
                             "com a versão atual na extensão 'versaoAtual'.")
            .ProducesProblem(StatusCodes.Status404NotFound)
            .ProducesProblem(StatusCodes.Status409Conflict);
        grupo.MapDelete("/{id:int}", Excluir).WithName("ExcluirTarefa").WithSummary("Exclui uma tarefa.")
            .ProducesProblem(StatusCodes.Status404NotFound);
        return grupo;
    }

    internal static async Task<Results<Ok<PaginaDto<TarefaResumoDto>>, ValidationProblem>> Listar(
        [AsParameters] ConsultaTarefasQuery consulta, ConsultaTarefasServico servico, HttpContext http,
CancellationToken ct)
    {
        var erros = new Dictionary<string, string[]>();
        if (consulta.OrdenarPor is { } campo && !ConsultaTarefasQuery.CamposOrdenacao.Contains(campo,
StringComparer.OrdinalIgnoreCase))
            erros["ordenarPor"] = [$"Campo de ordenação não permitido. Use: {string.Join(", ",
ConsultaTarefasQuery.CamposOrdenacao)}."];
        if (consulta.Direcao is { } dir && dir is not ("asc" or "desc"))
            erros["direcao"] = ["Use 'asc' ou 'desc'."];
        if (erros.Count > 0) return TypedResults.ValidationProblem(erros, title: "Parâmetros de consulta
inválidos.");

        var pagina = await servico.ConsultarAsync(consulta, ct);
        AdicionarCabecalhosPaginacao(http, pagina);
        return TypedResults.Ok(pagina);
    }

    internal static async Task<Results<Ok<TarefaDetalhesDto>, ProblemHttpResult>> Obter(int id,
ITarefaServico servico, CancellationToken ct) =>
        await servico.ObterAsync(id, ct) is { } t
            ? TypedResults.Ok(ConsultaTarefasServico.ParaDetalhes(t))
            : NaoEncontrada(id);

    internal static async Task<Results<Created<TarefaDetalhesDto>, ValidationProblem, ProblemHttpResult>>
Criar(
        CriarTarefaDto dto, ITarefaServico servico, TimeProvider relogio, CancellationToken ct)
    {
        if (ValidarRegras(dto.Prioridade, dto.Prazo, SituacaoTarefa.Pendente, relogio) is { } erro)
return erro;
        if (await servico.ExisteTituloAsync(dto.Titulo, null, ct)) return TituloDuplicado(dto.Titulo);
```

```
        var criada = await servico.CriarAsync(new Tarefa
        {
            Titulo = dto.Titulo.Trim(), Descricao = dto.Descricao, Prioridade = dto.Prioridade, Prazo =
dto.Prazo
        }, ct);
        return TypedResults.Created($"/api/v2/tarefas/{criada.Id}",
ConsultaTarefasServico.ParaDetalhes(criada));
    }

    internal static async Task<Results<Ok<TarefaDetalhesDto>, ValidationProblem, ProblemHttpResult>>
Atualizar(
        int id, AtualizarTarefaDto dto, ITarefaServico servico, TimeProvider relogio, HttpContext http,
CancellationToken ct)
    {
        if (ValidarRegras(dto.Prioridade, dto.Prazo, dto.Situacao, relogio) is { } erro) return erro;
        byte[] versao;
        try { versao = Convert.FromBase64String(dto.Versao); }
        catch (FormatException) { return TypedResults.ValidationProblem(new Dictionary<string, string[]>
{ ["versao"] = ["Versão em formato inválido."] }); }

        if (await servico.ObterAsync(id, ct) is null) return NaoEncontrada(id);
        if (await servico.ExisteTituloAsync(dto.Titulo, id, ct)) return TituloDuplicado(dto.Titulo);

        var tarefa = new Tarefa
        {
            Id = id, Titulo = dto.Titulo.Trim(), Descricao = dto.Descricao, Situacao = dto.Situacao,
            Prioridade = dto.Prioridade, Prazo = dto.Prazo, Versao = versao
        };
        try
        {
            await servico.AtualizarAsync(tarefa, http.User.Identity?.Name, ct);
        }
        catch (ConflitoConcorrenciaException ex)
        {
            if (ex.ValoresAtuais is null) return NaoEncontrada(id);
            return TypedResults.Problem(statusCode: StatusCodes.Status409Conflict, type: TipoConflito,
                title: "A tarefa foi alterada por outra requisição.",
                detail: "Obtenha a tarefa novamente, reaplique suas alterações e envie a nova versão.",
                extensions: new Dictionary<string, object?> { ["versaoAtual"] =
Convert.ToBase64String(ex.ValoresAtuais.Versao) });
        }
        return TypedResults.Ok(ConsultaTarefasServico.ParaDetalhes((await servico.ObterAsync(id, ct))!));
    }

    internal static async Task<Results<NoContent, ProblemHttpResult>> Excluir(int id, ITarefaServico
servico, CancellationToken ct) =>
        await servico.ExcluirAsync(id, ct) ? TypedResults.NoContent() : NaoEncontrada(id);

    // ---------- auxiliares ----------

    private static ValidationProblem? ValidarRegras(PrioridadeTarefa prioridade, DateOnly? prazo,
SituacaoTarefa situacao, TimeProvider relogio)
    {
        var hoje = DateOnly.FromDateTime(relogio.GetLocalNow().Date);
        if (prioridade == PrioridadeTarefa.Alta && prazo is null)
            return TypedResults.ValidationProblem(new Dictionary<string, string[]> { ["prazo"] =
["Tarefas de prioridade Alta precisam de prazo."] });
        if (prazo < hoje && situacao != SituacaoTarefa.Concluida)
            return TypedResults.ValidationProblem(new Dictionary<string, string[]> { ["prazo"] = ["O
prazo não pode estar no passado."] });
        return null;
    }

    private static ProblemHttpResult NaoEncontrada(int id) =>
        TypedResults.Problem(statusCode: 404, title: "Tarefa não encontrada.", detail: $"Não existe
tarefa com id {id}.");

    private static ProblemHttpResult TituloDuplicado(string titulo) =>
        TypedResults.Problem(statusCode: 409, type: TipoConflito, title: "Título já utilizado.",
            detail: $"Já existe uma tarefa com o título '{titulo.Trim()}'.");

    private static void AdicionarCabecalhosPaginacao<T>(HttpContext http, PaginaDto<T> pagina)
    {
        http.Response.Headers["X-Total-Count"] = pagina.TotalItens.ToString();
        var links = new List<string>();
        string Url(int p) => $"<{http.Request.Path}{QueryString.Create(http.Request.Query
            .Where(q => !q.Key.Equals("pagina", StringComparison.OrdinalIgnoreCase))
            .SelectMany(q => q.Value.Select(v => KeyValuePair.Create(q.Key, v)))
            .Append(KeyValuePair.Create("pagina", (string?)p.ToString())))}>";
        links.Add($"{Url(1)}; rel=\"first\"");
        if (pagina.Pagina > 1) links.Add($"{Url(pagina.Pagina - 1)}; rel=\"prev\"");
        if (pagina.Pagina < pagina.TotalPaginas) links.Add($"{Url(pagina.Pagina + 1)}; rel=\"next\"");
        if (pagina.TotalPaginas > 0) links.Add($"{Url(pagina.TotalPaginas)}; rel=\"last\"");
        http.Response.Headers.Link = string.Join(", ", links);
    }
```

```
}
```

**Arquivo:** `src/PortalTarefas.Web/Contratos/TarefasApiContratos.cs` (commit `65c1594`)

```
using System.ComponentModel;
using System.ComponentModel.DataAnnotations;
using Microsoft.AspNetCore.Mvc;
using PortalTarefas.Web.Modelos;

namespace PortalTarefas.Web.Contratos;

// DTOs da API v2. A entidade persistida (Tarefa) nunca é exposta diretamente.

/// <summary>Dados para cadastrar uma tarefa.</summary>
public sealed record CriarTarefaDto(
    [property: Required, StringLength(120, MinimumLength = 3), Description("Título único da tarefa.")]
string Titulo,
    [property: StringLength(1000)] string? Descricao = null,
    PrioridadeTarefa Prioridade = PrioridadeTarefa.Media,
    [property: Description("Data limite (AAAA-MM-DD). Obrigatória para prioridade Alta.")] DateOnly?
Prazo = null);

/// <summary>Dados para substituir uma tarefa existente (PUT). Exige a versão lida
anteriormente.</summary>
public sealed record AtualizarTarefaDto(
    [property: Required, StringLength(120, MinimumLength = 3)] string Titulo,
    [property: StringLength(1000)] string? Descricao,
    SituacaoTarefa Situacao,
    PrioridadeTarefa Prioridade,
    DateOnly? Prazo,
    [property: Required, Description("Token de concorrência devolvido no GET (Base64).")] string Versao);

/// <summary>Representação resumida usada em listagens.</summary>
public sealed record TarefaResumoDto(int Id, string Titulo, SituacaoTarefa Situacao, PrioridadeTarefa
Prioridade, DateOnly? Prazo,
    [property: Description("Verdadeiro quando o prazo já passou e a tarefa não foi concluída (campo
adicionado na revisão 2 do contrato).")] bool Vencida);

/// <summary>Representação completa de uma tarefa.</summary>
public sealed record TarefaDetalhesDto(int Id, string Titulo, string? Descricao, SituacaoTarefa Situacao,
    PrioridadeTarefa Prioridade, DateOnly? Prazo, DateTimeOffset CriadaEm,
    [property: Description("Enviar no PUT para detecção de edição concorrente.")] string Versao);

/// <summary>Página de resultados com metadados de navegação.</summary>
public sealed record PaginaDto<T>(IReadOnlyList<T> Itens, int Pagina, int Tamanho, int TotalItens, int
TotalPaginas);

/// <summary>Parâmetros de consulta da listagem (query string).</summary>
public sealed class ConsultaTarefasQuery
{
    public const int TamanhoMaximo = 50;
    public static readonly string[] CamposOrdenacao = ["id", "titulo", "prazo", "prioridade", "situacao",
"criadaEm"];

    // Propriedades anuláveis = parâmetros opcionais no contrato; os padrões ficam nas propriedades
*Efetiva.
    [FromQuery(Name = "pagina")]
    [Range(1, int.MaxValue, ErrorMessage = "A página deve ser maior ou igual a 1.")]
    [Description("Número da página (começa em 1)."), DefaultValue(1)]
    public int? Pagina { get; init; }

    [FromQuery(Name = "tamanho")]
    [Range(1, TamanhoMaximo, ErrorMessage = "O tamanho deve estar entre 1 e 50.")]
    [Description("Itens por página (1 a 50)."), DefaultValue(10)]
    public int? Tamanho { get; init; }

    [FromQuery(Name = "ordenarPor")]
    [Description("Campo de ordenação: id, titulo, prazo, prioridade, situacao ou criadaEm."),
DefaultValue("id")]
    public string? OrdenarPor { get; init; }

    [FromQuery(Name = "direcao")]
    [Description("asc ou desc."), DefaultValue("asc")]
    public string? Direcao { get; init; }

    [FromQuery(Name = "situacao")]
    public SituacaoTarefa? Situacao { get; init; }

    [FromQuery(Name = "prioridade")]
    public PrioridadeTarefa? Prioridade { get; init; }

    [FromQuery(Name = "busca")]
    [Description("Trecho do título.")]
    [StringLength(100)]
```

```
    public string? Busca { get; init; }

    [FromQuery(Name = "prazoAte")]
    [Description("Somente tarefas com prazo até esta data (inclusive).")]
    public DateOnly? PrazoAte { get; init; }

    public int PaginaEfetiva => Pagina ?? 1;
    public int TamanhoEfetivo => Tamanho ?? 10;
}
```

**Arquivo:** `src/PortalTarefas.Web/Servicos/ConsultaTarefasServico.cs` (commit `65c1594`)

```
using Microsoft.EntityFrameworkCore;
using PortalTarefas.Web.Contratos;
using PortalTarefas.Web.Dados;
using PortalTarefas.Web.Modelos;

namespace PortalTarefas.Web.Servicos;

/// <summary>
/// Leitura paginada da API: filtros, ordenação e paginação são aplicados no IQueryable,
/// portanto traduzidos para SQL (WHERE / ORDER BY / OFFSET FETCH) antes da materialização.
/// </summary>
public sealed class ConsultaTarefasServico(PortalTarefasDbContext db, TimeProvider relogio)
{
    public async Task<PaginaDto<TarefaResumoDto>> ConsultarAsync(ConsultaTarefasQuery q,
CancellationToken ct)
    {
        var consulta = db.Tarefas.AsNoTracking();
        var hoje = DateOnly.FromDateTime(relogio.GetLocalNow().Date);

        if (q.Situacao is { } situacao) consulta = consulta.Where(t => t.Situacao == situacao);
        if (q.Prioridade is { } prioridade) consulta = consulta.Where(t => t.Prioridade == prioridade);
        if (!string.IsNullOrWhiteSpace(q.Busca)) consulta = consulta.Where(t =>
t.Titulo.Contains(q.Busca));
        if (q.PrazoAte is { } prazoAte) consulta = consulta.Where(t => t.Prazo <= prazoAte);

        var total = await consulta.CountAsync(ct);
        var desc = string.Equals(q.Direcao, "desc", StringComparison.OrdinalIgnoreCase);

        // Lista permitida: cada campo aceito mapeia para uma expressão conhecida; Id desempata a ordem.
        IOrderedQueryable<Tarefa> ordenada = (q.OrdenarPor?.ToLowerInvariant() ?? "id") switch
        {
            "titulo" => desc ? consulta.OrderByDescending(t => t.Titulo) : consulta.OrderBy(t =>
t.Titulo),
            "prazo" => desc ? consulta.OrderByDescending(t => t.Prazo) : consulta.OrderBy(t => t.Prazo),
            "prioridade" => desc ? consulta.OrderByDescending(t => t.Prioridade) : consulta.OrderBy(t =>
t.Prioridade),
            "situacao" => desc ? consulta.OrderByDescending(t => t.Situacao) : consulta.OrderBy(t =>
t.Situacao),
            "criadaem" => desc ? consulta.OrderByDescending(t => t.CriadaEm) : consulta.OrderBy(t =>
t.CriadaEm),
            _ => desc ? consulta.OrderByDescending(t => t.Id) : consulta.OrderBy(t => t.Id),
        };

        var itens = await ordenada.ThenBy(t => t.Id)
            .Skip((q.PaginaEfetiva - 1) * q.TamanhoEfetivo).Take(q.TamanhoEfetivo)
            .Select(t => new TarefaResumoDto(t.Id, t.Titulo, t.Situacao, t.Prioridade, t.Prazo,
                t.Prazo < hoje && t.Situacao != SituacaoTarefa.Concluida))   // projeção
            .ToListAsync(ct);

        return new PaginaDto<TarefaResumoDto>(itens, q.PaginaEfetiva, q.TamanhoEfetivo, total,
            (int)Math.Ceiling(total / (double)q.TamanhoEfetivo));
    }

    public static TarefaDetalhesDto ParaDetalhes(Tarefa t) =>
        new(t.Id, t.Titulo, t.Descricao, t.Situacao, t.Prioridade, t.Prazo, t.CriadaEm,
Convert.ToBase64String(t.Versao));
}
```

**Arquivo:** `src/PortalTarefas.Frontend/nswag.json` (commit `65c1594`)

```
{
  "runtime": "Net100",
  "documentGenerator": {
    "fromDocument": {
      "url": "../../docs/openapi/portal-tarefas_v2.json"
    }
  },
  "codeGenerators": {
    "openApiToTypeScriptClient": {
      "className": "PortalTarefasClient",
      "operationGenerationMode": "SingleClientFromOperationId",
```

```
      "template": "Fetch",
      "typeScriptVersion": 5.0,
      "typeStyle": "Interface",
      "dateTimeType": "String",
      "nullValue": "Undefined",
      "generateClientClasses": true,
      "generateDtoTypes": true,
      "generateOptionalParameters": true,
      "useAbortSignal": true,
      "wrapDtoExceptions": false,
      "exceptionClass": "ApiException",
      "markOptionalProperties": true,
      "enumStyle": "StringLiteral",
      "output": "src/api/gerado/portal-tarefas-client.ts"
    }
  }
}
```

**Arquivo:** `src/PortalTarefas.Frontend/scripts/chamada-real.ts` (commit `65c1594`)

```
// Chamada real à API usando somente o cliente gerado pelo NSwag (sem DTOs escritos à mão).
// Uso: API_URL=https://localhost:7080 npx tsx scripts/chamada-real.ts
import { PortalTarefasClient, ApiException, type ProblemDetails } from '../src/api/gerado/portal-tarefas-
client';

const cliente = new PortalTarefasClient(process.env.API_URL ?? 'https://localhost:7080', { fetch:
globalThis.fetch.bind(globalThis) });

const pagina = await cliente.listarTarefas(1, 3, 'prazo', 'asc', 'Pendente');
console.log(`Página ${pagina.pagina}/${pagina.totalPaginas} — ${pagina.totalItens} tarefas pendentes`);
for (const t of pagina.itens) console.log(`  #${t.id} ${t.titulo} (prazo ${t.prazo ?? '—'})`);

const criada = await cliente.criarTarefa({ titulo: `Criada pelo cliente TS ${Date.now()}`, prioridade:
'Baixa' });
console.log('Criada:', criada.id, criada.titulo, 'versão', criada.versao);

try {
  await cliente.obterTarefa(999999);
} catch (e) {
  // Para respostas documentadas (404 → ProblemDetails), o cliente gerado lança o próprio objeto tipado.
  if (e instanceof ApiException) console.log('Erro não documentado:', e.status);
  else { const p = e as ProblemDetails; console.log('Erro esperado (Problem Details):', p.status,
p.title); }
}
```

### Configuração do OpenAPI

```
// Program.cs (trecho): documento OpenAPI v2 e UI somente em Development
builder.Services.AddOpenApi("v2", o => o.AddDocumentTransformer((doc, _, _) => { ... })
    .ShouldInclude = d => d.RelativePath?.StartsWith("api/v2") == true);
builder.Services.Configure<OpenApiOptions>("v2", o => o.CreateSchemaReferenceId = tipo => /*
PaginaTarefaResumo */ ...);
app.MapTarefasApiV2();
app.MapOpenApi();                                        // /openapi/v2.json
if (app.Environment.IsDevelopment()) app.MapScalarApiReference(...);
// PortalTarefas.Web.csproj: <OpenApiDocumentsDirectory>../../docs/openapi</OpenApiDocumentsDirectory>
```

## Apêndice B – Evidências de execução

### Sucesso com metadados, página vazia e limite mínimo

```
### Listagem padrão (sucesso, com X-Total-Count e Link)
$ curl https://localhost:7080/api/v2/tarefas?tamanho=3
HTTP/2 200
content-type: application/json; charset=utf-8
x-total-count: 8
link: </api/v2/tarefas?tamanho=3&pagina=1>; rel="first", </api/v2/tarefas?tamanho=3&pagina=2>; rel="next",
</api/v2/tarefas?tamanho=3&pagina=3>; rel="last"
x-correlation-id: 142bbb9ff8934b9fb495309080ebf1fa
{
    "itens": [
        {
            "id": 1,
            "titulo": "Configurar ambiente .NET 10",
            "situacao": "Concluida",
            "prioridade": "Alta",
            "prazo": "2026-10-30",
            "vencida": false
        },
        {
            "id": 2,
            "titulo": "Montar pipeline HTTP",
```

```
            "situacao": "EmAndamento",
            "prioridade": "Media",
            "prazo": "2026-11-07",
            "vencida": true
        },
        {
            "id": 3,
            "titulo": "Escrever relatório 1",
            "situacao": "Pendente",
            "prioridade": "Baixa",
            "prazo": "2026-11-16",
            "vencida": false
        }
    ],
```

### 404 e 201 com Location

```
### GET por id inexistente (404)
$ curl https://localhost:7080/api/v2/tarefas/999
HTTP/2 404
content-type: application/problem+json
x-correlation-id: 8148ff526a5542fa80b1fce217737e6c
{
    "type": "https://tools.ietf.org/html/rfc9110#section-15.5.5",
    "title": "Tarefa não encontrada.",
    "status": 404,
    "detail": "Não existe tarefa com id 999.",
    "instance": "GET /api/v2/tarefas/999",
    "traceId": "00-efab1b6cb589fa84fc2f87035a438559-1aefa918c97a3fae-00",
    "correlationId": "8148ff526a5542fa80b1fce217737e6c"
}

### POST válido (201 + Location)
$ curl -X POST -H Content-Type: application/json -d {"titulo":"Documentar API
REST","prioridade":"Media","prazo":"2026-12-01"} https://localhost:7080/api/v2/tarefas
HTTP/2 201
content-type: application/json; charset=utf-8
location: /api/v2/tarefas/9
x-correlation-id: 32fdf99db35b4d59bebf5502a53e45c2
{
    "id": 9,
    "titulo": "Documentar API REST",
    "descricao": null,
    "situacao": "Pendente",
    "prioridade": "Media",
    "prazo": "2026-12-01",
    "criadaEm": "2026-11-09T01:36:22.7511165+00:00",
    "versao": "AAAAAAAAB9k="
```

### 409 por título duplicado e por versão desatualizada

```
### POST conflito (título duplicado → 409)
$ curl -X POST -H Content-Type: application/json -d {"titulo":"Montar pipeline HTTP"}
https://localhost:7080/api/v2/tarefas
HTTP/2 409
content-type: application/problem+json
x-correlation-id: 4f236ea57088465db1ed40e301a29d33
{
    "type": "https://portaltarefas.example/problemas/conflito",
    "title": "Título já utilizado.",
    "status": 409,
    "detail": "Já existe uma tarefa com o título 'Montar pipeline HTTP'.",
    "instance": "POST /api/v2/tarefas",
    "traceId": "00-406caa07e94d0cc26bea1a7d2e15aebf-c28a6b578b71eb7b-00",
    "correlationId": "4f236ea57088465db1ed40e301a29d33"
}

### PUT com versão atual (200)
$ curl -X PUT -H Content-Type: application/json -d {"titulo":"Testar
concorrência","situacao":"EmAndamento","prioridade":"Alta","prazo":"2026-11-30","versao":"AAAAAAAAB9c="} h …
HTTP/2 200
content-type: application/json; charset=utf-8
x-correlation-id: 6b0f55edaa7f416cba568066f3ada189
{
    "id": 7,
    "titulo": "Testar concorrência",
    "descricao": null,
    "situacao": "EmAndamento",
    "prioridade": "Alta",
    "prazo": "2026-11-30",
    "criadaEm": "2026-11-09T01:36:18.6983832+00:00",
    "versao": "AAAAAAAAB9o="
}

### PUT repetido com a versão antiga (409 conflito de concorrência)
$ curl -X PUT -H Content-Type: application/json -d {"titulo":"Testar
concorrência","situacao":"Concluida","prioridade":"Alta","prazo":"2026-11-30","versao":"AAAAAAAAB9c="} htt …
HTTP/2 409
content-type: application/problem+json
x-correlation-id: abb335cf156f450581111f854bd47d96
```

```
{
    "type": "https://portaltarefas.example/problemas/conflito",
    "title": "A tarefa foi alterada por outra requisição.",
    "status": 409,
```

### Mudança compatível e regeneração do cliente

```
== Mudança compatível no contrato: novo campo de saída 'vencida' em TarefaResumoDto ==
(adicionar campo em resposta não quebra clientes existentes; nenhum campo removido/renomeado)

$ dotnet build src/PortalTarefas.Web   # regenera docs/openapi/portal-tarefas_v2.json
Build succeeded.

$ git diff docs/openapi/portal-tarefas_v2.json
diff --git a/PortalTarefas/docs/openapi/portal-tarefas_v2.json b/PortalTarefas/docs/openapi/portal-
tarefas_v2.json
index c316f87..871627c 100644
--- a/PortalTarefas/docs/openapi/portal-tarefas_v2.json
+++ b/PortalTarefas/docs/openapi/portal-tarefas_v2.json
@@ -627,7 +627,8 @@
           "titulo",
           "situacao",
           "prioridade",
-          "prazo"
+          "prazo",
+          "vencida"
         ],
         "type": "object",
         "properties": {
@@ -650,6 +651,10 @@
               "string"
             ],
             "format": "date"
+          },
+          "vencida": {
+            "type": "boolean",
+            "description": "Verdadeiro quando o prazo já passou e a tarefa não foi concluída (campo adicionado
na revisão 2 do contrato)."
           }
         },
         "description": "Representação resumida usada em listagens."

$ npm run gerar-cliente
NSwag NPM CLI
Found runtime from config: Net100
Using runtime: Net100
NSwag command line tool for .NET Core Net100, toolchain v14.7.1.0 (NJsonSchema v11.6.1.0 (Newtonsoft.Json
v13.0.0.0))
Visit http://NSwag.org for more information.
NSwag bin directory: /home/user/projetos/PortalTarefas/src/PortalTarefas.Frontend/node_modules/nswag/bin/
binaries/Net100
Executing file 'nswag.json' with variables ''...
Done.
Duration: 00:00:00.6153064

$ git diff src/PortalTarefas.Frontend/src/api/gerado/portal-tarefas-client.ts
diff --git a/PortalTarefas/src/PortalTarefas.Frontend/src/api/gerado/portal-tarefas-client.ts
b/PortalTarefas/src/PortalTarefas.Frontend/src/api/gerado/portal-tarefas-cli …
index 3f88bea..a4a344d 100644
--- a/PortalTarefas/src/PortalTarefas.Frontend/src/api/gerado/portal-tarefas-client.ts
+++ b/PortalTarefas/src/PortalTarefas.Frontend/src/api/gerado/portal-tarefas-client.ts
@@ -423,6 +423,8 @@ export interface TarefaResumoDto {
     situacao: SituacaoTarefa;
     prioridade: PrioridadeTarefa;
     prazo: string | undefined;
+    /** Verdadeiro quando o prazo já passou e a tarefa não foi concluída (campo adicionado na revisão 2 do
contrato). */
+    vencida: boolean;

     [key: string]: any;
 }

$ npx tsc --noEmit -p tsconfig.app.json   # código existente continua compilando
```

### SQL gerado: paginação aplicada no banco

```
Executed DbCommand (15ms) [Parameters=[@hoje='?' (DbType = Date), @p='?' (DbType = Int32), @p1='?' (DbType =
Int32)], CommandType='Text', CommandTimeout='30'] SELECT [t].[Id], [t].[Titulo], [t].[Situacao], [t].
[Prioridade], [t].[Prazo], CASE     WHEN CASE         WHEN [t].[Prazo] < @hoje THEN CAST(1 AS bit)         ELSE
CAST(0 AS bit)     END = CAST(1 AS bit) AND [t].[Situacao] <> N'Concluida' THE …
```

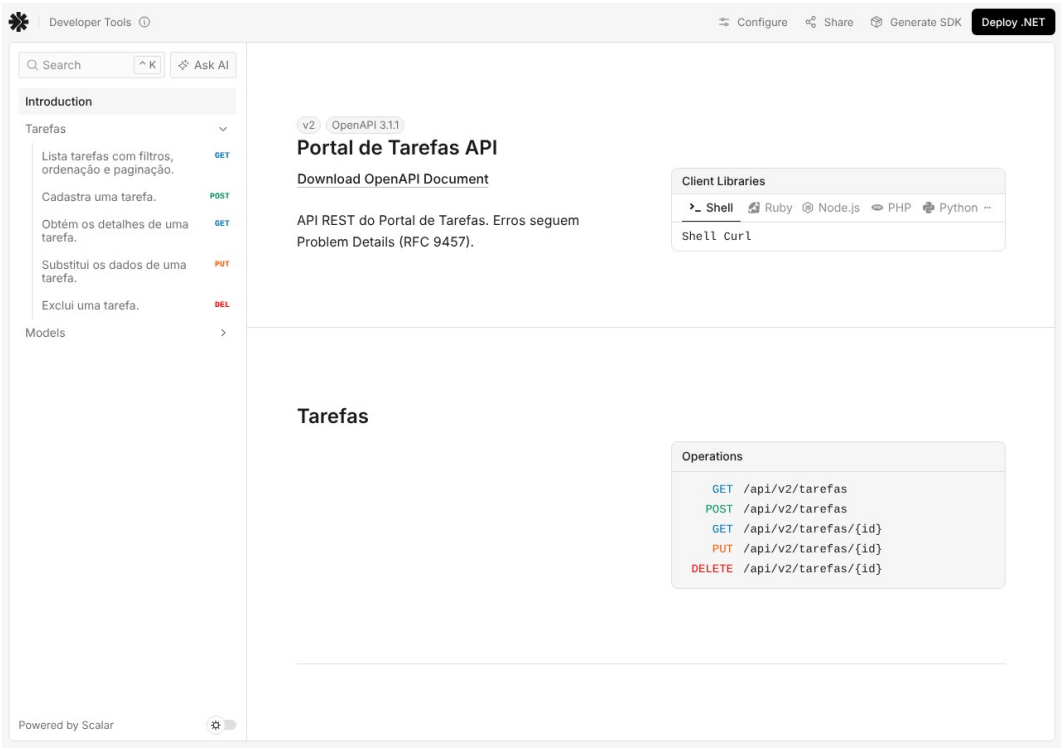



**Figura B1** – Interface interativa Scalar (somente Development) lendo /openapi/v2.json